OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Stack: Theory, Operations & Expressions

LIFO Principle, Python Implementations, Polish Notations, Call Stack & Shunting-Yard

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B1: Topic 2 (Class 12)	Data Structures: Stack as an Abstract Data Type (ADT), LIFO principle, linear data structures, applications of stacks.	LIFO insertion and deletion at a single logical end (Top). Identification of stack applications: expression evaluation, parenthesis balancing, backtracking.
Section B1: Topic 2 (Class 12)	Stack Operations & Implementations: `push()`, `pop()`, `peek()`, `is_empty()`, `size()`, stack overflow and underflow conditions.	Python list implementation using `append()` and `pop()`. Underflow detection (`len == 0`), index 0 vs index −1 as stack top complexity traps.
Section B1: Topic 2 (Class 12)	Expression Notations & Conversion: Infix, Prefix, Postfix notations, conversion of infix to postfix using stack, evaluation of postfix expressions.	Operator precedence, right-to-left associativity of exponentiation (**), operand popping order during evaluation (`first_pop = op2`, `second_pop = op1`).

EXAM PATTERN

CUET UG Exam Architecture: Unit 2 (Stack) accounts for approximately 10% to 14% of Section B1 in the CUET UG Computer Science paper (4 to 6 MCQs). High-frequency question archetypes include tracing stack states during infix-to-postfix conversion, calculating numeric values from postfix expressions, identifying stack overflow/underflow boundary invariants, and analyzing algorithmic time complexity of list vs deque stack implementations.

Concept Module 1: Stack as an Abstract Data Type (ADT)

1.1 The LIFO Principle & Architectural Definition

A **Stack** is a restricted linear data structure that adheres strictly to the **Last-In, First-Out (LIFO)** access discipline. In a stack:

- Elements are inserted and removed exclusively from one logical boundary, designated as the **Top of Stack (Top)**.
- The opposite boundary, where elements are immobilized until all overlying items are removed, is termed the **Base (or Bottom)**.
- A stack is an **Abstract Data Type (ADT)** because its formal specification defines **what** operations are supported (`push`, `pop`, `peek`, `is_empty`), entirely decoupling behavior from the underlying physical storage mechanism (contiguous array, dynamic vector, or pointer-based linked list).

1.2 Stack Boundary Invariants: Overflow and Underflow

A stack is characterized by two fundamental boundary conditions:

1. **Stack Underflow:** Occurs when an extraction operation (`pop()` or `peek()`) is attempted on a stack that contains zero elements (Top $= -1$ or `len(s) == 0`). In Python, attempting `list.pop()` on an empty list immediately raises `IndexError: pop from empty list`. Robust implementations enforce defensive underflow guard clauses before attempting element access.
2. **Stack Overflow:** Occurs when an insertion operation (`push()`) is attempted on a bounded stack that has reached its maximum allocated capacity (Top $=$ Max $- 1$).
   - In languages with fixed-size static arrays (such as C/C++ or Java), overflow is a persistent operational hazard.
   - In standard Python, lists are **dynamically sized arrays**; a Python list automatically requests additional heap allocations when full, meaning physical stack overflow only occurs when host physical memory (RAM) is entirely exhausted. However, bounded stacks implemented for CUET problem models enforce an explicit `MAX_SIZE` boundary.

Concept Module 2: Stack Primitive Operations & Complexity

2.1 The Core Operational Suite

The complete canonical stack ADT interface comprises five fundamental methods:

1. `push(item)`: Inserts element `item` onto the top of the stack. If the stack is bounded and full, raises Stack Overflow. Increments the logical Top pointer (Top $\rightarrow$ Top $+ 1$).
2. `pop()`: Removes and returns the element currently residing at the top of the stack. If empty, raises Stack Underflow. Decrements the logical Top pointer (Top $\rightarrow$ Top $- 1$).
3. `peek()` / `top()`: Inspects and returns the element currently at the top of the stack **without** mutating or removing it. Raises Underflow if empty.
4. `is_empty()`: Predicate function returning Boolean `True` if the stack contains zero elements (Top $= -1$), otherwise `False`.
5. `size()`: Returns the total count of active elements currently stored on the stack.

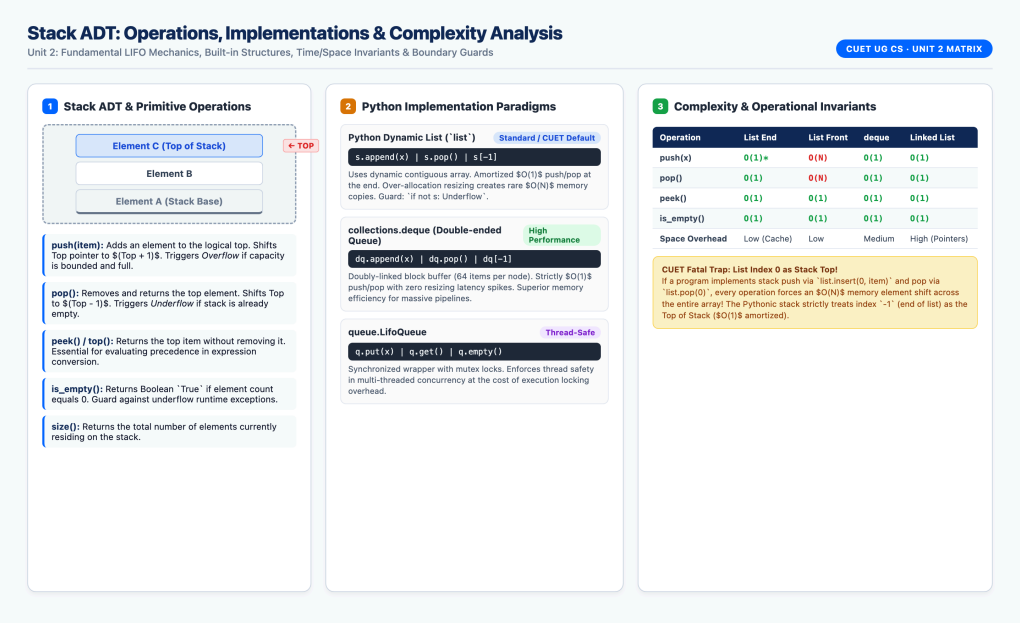


Figure 1: Figure 1: Stack Operations, Implementations & Complexity Matrix: Comparative analysis of primitive operations, Python internal data structures, and operational boundary invariants.

2.2 Time & Space Complexity Analysis

- **Time Complexity:** Every primitive stack operation (`push`, `pop`, `peek`, `is_empty`, `size`) executes in strictly $O(1)$ constant time under optimal implementations. In a Python list, `append()` achieves **amortized** $O(1)$ time because occasional array over-allocation resizing takes $O(N)$ memory transfer time, but occurs infrequently enough that average execution cost remains $O(1)$.
- **Space Complexity:** Auxiliary space is strictly $O(N)$, proportional to the number of elements N stored on the stack.

Concept Module 3: Python Implementation Paradigms

3.1 Python Dynamic List (`list`) Implementation

The standard and most ubiquitous method to implement a stack in Python is utilizing the built-in dynamic array `list`. Python lists offer high cache locality because elements are stored in contiguous memory addresses.

```
class ListStack:
    def __init__(self, capacity=None):
        self._items = []
        self._capacity = capacity

    def is_empty(self):
        return len(self._items) == 0

    def push(self, item):
        if self._capacity is not None and len(self._items) >= self._capacity:
            raise OverflowError("Stack Overflow: Maximum capacity reached.")
        self._items.append(item)  # O(1) amortized

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack Underflow: Attempted pop from empty stack.")
        return self._items.pop()   # O(1) removes last element

    def peek(self):
        if self.is_empty():
            raise IndexError("Stack Underflow: Attempted peek on empty stack.")
        return self._items[-1]     # O(1) inspects last element

    def size(self):
        return len(self._items)
```

The $O(N)$ List-Front Inversion Trap

A catastrophic algorithmic error frequently tested in CUET is utilizing `list.insert(0, item)` for `push` and `list.pop(0)` for `pop`.

- Because Python lists are contiguous arrays, inserting or deleting at index 0 requires the runtime engine to shift all N subsequent elements by one memory word.
- This degrades both `push` and `pop` from $O(1)$ to $O(N)$ linear time!
- **Strict Rule:** The top of a list-based stack **must always** correspond to the end of the list (`index = -1` or `len - 1`).

3.2 Double-Ended Queue (`collections.deque`) Implementation

When strict constant-time performance without resizing latency spikes is mandated, `collections.deque` is the industry standard.

- Internally implemented as a doubly-linked list of fixed-size blocks (typically 64 elements per block).
- Appending and popping at either extremity (`append()` and `pop()`) is guaranteed strictly $O(1)$ worst-case time with zero contiguous array reallocation overhead.
- Unlike lists, `deque` does not support random slice indexing, but for pure stack LIFO operations, it is computationally superior.

3.3 Synchronized Thread-Safe Stack (`queue.LifoQueue`)

In concurrent, multi-threaded server environments, `queue.LifoQueue` provides thread-safe stack access:

- Methods include `put(item)` (push), `get()` (pop), `qsize()`, and `empty()`.
- Implements internal synchronization locks (mutexes) around every mutation, preventing race conditions when multiple threads access the stack simultaneously.
- Mutex contention incurs execution overhead; hence, for single-threaded CUET algorithmic problems, `list` or `collections.deque` is preferred.

3.4 Custom Linked Stack (Singly-Linked Nodes)

A stack can also be implemented using dynamic pointer-based node objects. In a linked stack:

- Each `Node` encapsulates a `data` field and a `next` pointer pointing to the node below it.
- The `head` pointer of the linked list serves directly as the Top of the stack.
- `push` prepends a new node at the head ($O(1)$).
- `pop` advances the head pointer to `head.next` ($O(1)$).
- Advantage: Memory is allocated dynamically per element; no contiguous memory block is required.
- Disadvantage: Higher memory overhead due to storing explicit pointer references for every node.

Concept Module 4: Expression Notations & Algebraic Formats

4.1 Infix Notation

- **Syntax:** The binary operator is positioned **between** its two operands: Operand_1 Operator Operand_2 (e.g., $A + B$).
- **Characteristics:** This is the conventional human algebraic representation.
- **Limitation for Machines:** Ambiguous without explicit operator precedence hierarchies, associativity rules, and grouping parentheses. Evaluating an infix string directly requires multiple recursive scans or parenthetical unwrapping.

4.2 Prefix Notation (Polish Notation)

- **Syntax:** The operator precedes both of its operands: Operator Operand_1 Operand_2 (e.g., $+AB$).
- **Origin:** Invented by Polish mathematician Jan Łukasiewicz in 1924.
- **Characteristics:** Completely eliminates the need for parentheses and precedence rules. An expression is parsed deterministically without ambiguity.

4.3 Postfix Notation (Reverse Polish Notation - RPN)

- **Syntax:** The operator follows both of its operands: Operand_1 Operand_2 Operator (e.g., $AB+$).
- **Significance in Computing:** Postfix notation is the fundamental machine execution format utilized by stack-based virtual machines, compilers, and Hewlett-Packard scientific calculators.
- **Core Advantage:** Can be evaluated in a single linear pass from left to right using a single operand stack without ever looking ahead, backtracking, or checking parentheses.

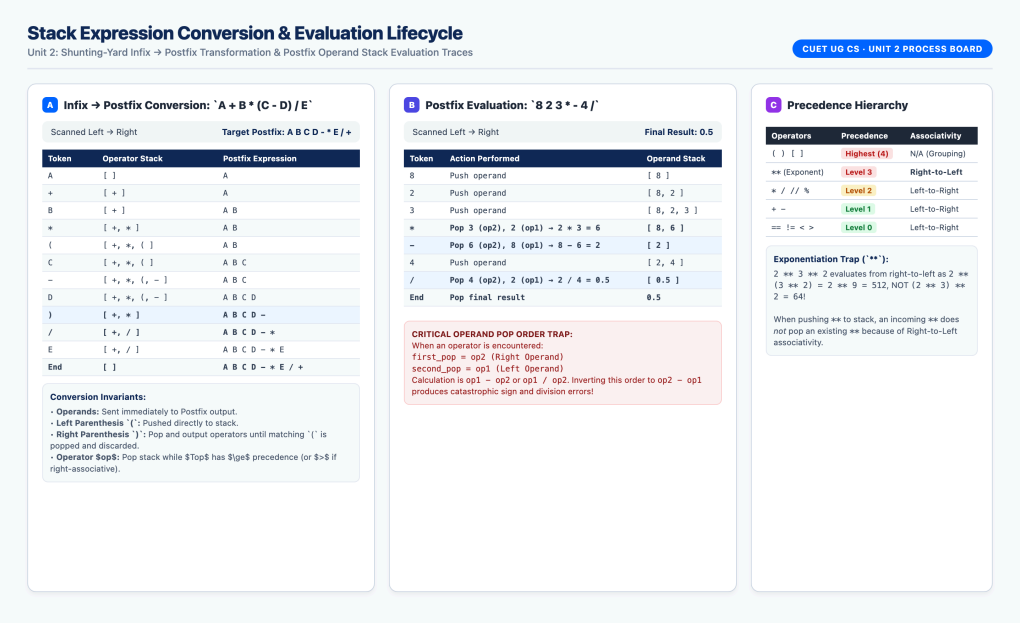


Figure 2: Figure 2: Stack Expression Conversion & Evaluation Lifecycle: Algorithmic trace of Shunting-Yard Infix-to-Postfix transformation and Postfix operand stack evaluation.

Concept Module 5: Shunting-Yard Infix to Postfix Algorithm

5.1 Operator Precedence and Associativity Table

To transform an infix string into postfix deterministically, the algorithm utilizes the standard mathematical precedence hierarchy:

Operator	Precedence	Associativity	CUET Conversion Rule
`( ) [ ]`	Highest (4)	None (Grouping)	`(` pushed to stack; `)` pops until `(` is reached
** (Exponent)	Level 3	**Right-to-Left**	Incoming ** does **not** pop top ** of equal precedence
`* / // %`	Level 2	Left-to-Right	Pops top operators with precedence ≥ 2
`+ -`	Level 1	Left-to-Right	Pops top operators with precedence ≥ 1
`== != < >`	Level 0	Left-to-Right	Relational operators evaluated after arithmetic

5.2 Step-by-Step Shunting-Yard Algorithm

Devised by Edsger Dijkstra, the Shunting-Yard algorithm operates linearly using an **Operator Stack** and an **Output Buffer**:

1. Initialize an empty operator stack and an empty output string/list.
2. Scan the infix token stream from **Left to Right**:
   - **If Token is an Operand (variable or number):** Append it immediately to the output string.
   - **If Token is a Left Parenthesis `(`:** Push it directly onto the operator stack.
   - **If Token is a Right Parenthesis `)`:** Repeatedly pop operators from the stack and append them to the output until a matching `(` is encountered on top of the stack. Pop and discard the `(` (parentheses never appear in the postfix output).
   - **If Token is an Operator `op1`:**
     - While the stack is not empty, and the top element is not '(':
       - Either top operator `op2` has strictly greater precedence than `op1`,
       - OR `op2` has equal precedence to `op1` and `op1` is Left-Associative:
       - Pop `op2` from the stack and append it to the output string.
     - Push the incoming operator `op1` onto the stack.
3. **At End of Infix Stream:** Pop all remaining operators from the stack one by one and append them to the output string.

Concept Module 6: Postfix & Prefix Evaluation Algorithms

6.1 Linear Postfix Evaluation Algorithm

Evaluating a postfix expression (RPN) is one of the most reliable and computationally elegant stack applications:

1. Initialize an empty **Operand Stack**.
2. Scan the postfix tokens from **Left to Right**:
   - **If Token is an Operand:** Push its numeric value onto the operand stack.
   - **If Token is an Operator `op`:**
     - Pop the top element from the operand stack and assign it to op_2 (**Right Operand**).
     - Pop the next top element from the operand stack and assign it to op_1 (**Left Operand**).
     - Compute the intermediate result: $\text{res} = \text{op}_1 \ \ \text{op} \ \ \text{op}_2$.
     - Push res back onto the operand stack.
3. **At End of Token Stream:** The operand stack will contain exactly one single value, which is the final evaluated result of the entire expression.

The Non-Commutative Operand Inversion Hazard

In arithmetic operations where order matters (subtraction `-`, division `/`, floor division `//`, modulus `%`, and exponentiation `**`), reversing operands is fatal:

- First popped element **must be** the second operand (op_2).
- Second popped element **must be** the first operand (op_1).
- Correct: $\text{op}_1 - \text{op}_2$ and $\frac{\text{op}_1}{\text{op}_2}$.
- **Fatal Trap:** Computing $\text{op}_2 - \text{op}_1$ yields incorrect negative or inverted values!

6.2 Prefix Expression Evaluation Algorithm

Prefix expressions are evaluated similarly, but with a crucial directional distinction:

1. Scan the prefix token stream from **Right to Left** (reverse scan).
2. **If Token is an Operand:** Push onto the operand stack.
3. **If Token is an Operator `op`:**
   - Pop the first operand from the stack → this is op_1 (**Left Operand**).
   - Pop the second operand from the stack → this is op_2 (**Right Operand**).
   - Compute $\text{res} = \text{op}_1 \ \ \text{op} \ \ \text{op}_2$ and push res back onto the stack.
4. When scan terminates, the single remaining value on the stack is the final result.

Concept Module 7: Core Stack Applications & Systems Mechanics

7.1 Balanced Delimiter & Parentheses Validation

In compilers and syntax linters, stacks verify that code blocks and bracket pairs are properly nested:

- Algorithm: Maintain an empty stack. Scan character stream.
  - When an opening delimiter (`(`, `[`, `{`) is seen, push it.
  - When a closing delimiter (`)`, `]`, `}`) is seen:
    - If stack is empty, return `False` (Unmatched closing delimiter / Underflow).
    - Pop the top opening delimiter. If it does not match the closing type (e.g. `[` popped against `)`), return `False` (Mismatched delimiter).
  - At stream termination: If stack is empty, return `True` (Balanced); if non-empty, return `False` (Unclosed opening delimiter).

7.2 Function Call Stack, Activation Records & Recursion

Every running program relies on the hardware-mediated **Call Stack**:

- **Stack Frame (Activation Record):** When a function is invoked, the operating system allocates a stack frame storing:
  1. Return address in caller code.
  2. Formal parameter bindings.
  3. Local variables and temporary storage.

- **Recursion Mechanics:** Recursive invocations push successive stack frames onto the call stack. Base cases halt recursive pushes, triggering successive frame pops during the unwinding phase.
- **Call Stack Overflow & Python Recursion Limit:**
  - Python protects against runaway infinite recursion via `sys.getrecursionlimit()` (default 1000 frames).
  - Exceeding this limit triggers `RecursionError: maximum recursion depth exceeded`. Developers can query or adjust this via `sys.setrecursionlimit(n)`.

7.3 Backtracking, Undo Buffers & Graph Depth-First Search (DFS)

- **Browser Navigation:** Two stacks manage page history: `Back_Stack` and `Forward_Stack`. Clicking "Back" pops from `Back_Stack` and pushes to `Forward_Stack`. Navigating to a new link clears `Forward_Stack`.
- **Text Editor Undo/Redo:** Actions are pushed to `Undo_Stack`. Typing "Ctrl+Z" pops the last action, reverses it, and pushes to `Redo_Stack`.
- **Graph DFS Traversal:** Depth-First Search utilizes an explicit stack (or implicit recursion call stack) to visit nodes along deep paths before backtracking.

Concept Module 8: Comprehensive Worked Examples

WORKED EXAMPLE 1: INFIX TO POSTFIX CONVERSION WITH EXPONENTIATION

Convert the infix expression `P + Q * (R - S) ** T / U` to postfix notation using the standard operator stack algorithm.
Step-by-Step Conversion Trace:

1. `P` → Output: `P` | Stack: `[]`
2. `+` → Output: `P` | Stack: `[+]`
3. `Q` → Output: `P Q` | Stack: `[+]`
4. `*` → Output: `P Q` | Stack: `[+, *]`
5. `(` → Output: `P Q` | Stack: `[+, *, (]`
6. `R` → Output: `P Q R` | Stack: `[+, *, (]`
7. `-` → Output: `P Q R` | Stack: `[+, *, (, -]`
8. `S` → Output: `P Q R S` | Stack: `[+, *, (, -]`
9. `)` → Pop until `(`: Output: `P Q R S -` | Stack: `[+, *]`
10. `**` → Has higher precedence than `*`, push: Output: `P Q R S -` | Stack: `[+, *, **]`
11. `T` → Output: `P Q R S - T` | Stack: `[+, *, **]`
12. `/` → Top is `**` (precedence $3 > 2$), pop `**`; next top is `*` (precedence $2 == 2$, left-assoc), pop `*`. Push `/`: Output: `P Q R S - T ** *` | Stack: `[+, /]`
13. `U` → Output: `P Q R S - T ** * U` | Stack: `[+, /]`
14. End of stream → Pop `/`, pop `+`: **Final Postfix:** `P Q R S - T ** * U / +`

WORKED EXAMPLE 2: POSTFIX ARITHMETIC EVALUATION TRACE

Evaluate the postfix expression: `12 4 2 / + 5 3 * - 2 **` showing stack contents at every operator.
Step-by-Step Evaluation Trace:

- Push operands `12`, `4`, `2` → Stack: `[12, 4, 2]`
- Token `/`: Pop `op2 = 2`, `op1 = 4`. Compute `4 / 2 = 2.0`. Push `2.0` → Stack: `[12, 2.0]`
- Token `+`: Pop `op2 = 2.0`, `op1 = 12`. Compute `12 + 2.0 = 14.0`. Push `14.0` → Stack: `[14.0]`
- Push operands `5`, `3` → Stack: `[14.0, 5, 3]`
- Token `*`: Pop `op2 = 3`, `op1 = 5`. Compute `5 * 3 = 15.0`. Push `15.0` → Stack: `[14.0, 15.0]`
- Token `-`: Pop `op2 = 15.0`, `op1 = 14.0`. Compute `14.0 - 15.0 = -1.0`. Push `-1.0` → Stack: `[-1.0]`
- Push operand `2` → Stack: `[-1.0, 2]`
- Token `**`: Pop `op2 = 2`, `op1 = -1.0`. Compute `(-1.0) ** 2 = 1.0`. Push `1.0` → Stack: `[1.0]`
- **Final Result:** `1.0`

WORKED EXAMPLE 3: DELIMITER MATCHING VALIDATOR IMPLEMENTATION

Write a Python function `is_balanced(expression)` that validates whether all parentheses `()`, brackets `[]`, and curly braces `{}` in string `expression` are properly matched and closed.

```
def is_balanced(expression):
    matching = {")": "(", "]": "[", "}": "{"}
```

```
    stack = []
    for char in expression:
        if char in "([{":
            stack.append(char)
        elif char in ")]}":
            if not stack or stack[-1] != matching[char]:
                return False
            stack.pop()
    return len(stack) == 0

# Test cases
print(is_balanced("{[a + b] * (c - d)}"))  # True
print(is_balanced("([)]"))                 # False (Mismatched nesting)
print(is_balanced("((()"))                 # False (Unclosed opener)
```

WORKED EXAMPLE 4: STACK WITH O(1) GET-MINIMUM FUNCTIONALITY

Design a data structure `MinStack` supporting `push()`, `pop()`, `top()`, and retrieving the minimum element `get_min()` all in strictly $O(1)$ constant time.

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val):
        self.stack.append(val)
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self):
        if not self.stack:
            raise IndexError("Pop from empty MinStack")
        val = self.stack.pop()
        if val == self.min_stack[-1]:
            self.min_stack.pop()
        return val

    def top(self):
        return self.stack[-1]

    def get_min(self):
        return self.min_stack[-1]
```

Analysis: Maintaining a secondary parallel stack `min_stack` that records current minimums ensures $O(1)$ retrieval with $O(N)$ extra memory.

WORKED EXAMPLE 5: IN-PLACE TWO STACKS IN A SINGLE LIST ARRAY

Implement two distinct stacks `Stack1` and `Stack2` within a single fixed-size list `arr` of size N such that neither overflows unless the total combined number of elements exceeds N.

```
class TwoStacks:
    def __init__(self, n):
        self.size = n
        self.arr = [None] * n
        self.top1 = -1
        self.top2 = n

    def push1(self, x):
        if self.top1 < self.top2 - 1:
            self.top1 += 1
            self.arr[self.top1] = x
        else:
            raise OverflowError("Stack 1 Overflow")

    def push2(self, x):
        if self.top1 < self.top2 - 1:
            self.top2 -= 1
            self.arr[self.top2] = x
        else:
            raise OverflowError("Stack 2 Overflow")

    def pop1(self):
        if self.top1 >= 0:
            x = self.arr[self.top1]
            self.top1 -= 1
            return x
        raise IndexError("Stack 1 Underflow")

    def pop2(self):
        if self.top2 < self.size:
            x = self.arr[self.top2]
            self.top2 += 1
```

```
        return x
    raise IndexError("Stack 2 Underflow")
```

Concept Module 9: Advanced Stack Algorithms & Memory Mechanics

9.1 Infix to Prefix Conversion Algorithms

While Infix to Postfix is the most common compiler pipeline, converting Infix to Prefix (Polish Notation) is equally high-yield in competitive examinations.

Method 1: The Reversal & Postfix Inversion Protocol

1. Reverse the input Infix string from right to left.
2. Invert all parentheses: replace every `(` with `)` and every `)` with `(`.
3. Apply the standard Shunting-Yard algorithm to obtain the Postfix expression, with one critical modification:
   - For operators of equal precedence, the incoming operator does **not** pop the top operator (associativity is inverted from left to right).
4. Reverse the resulting Postfix expression string. The final string is the exact Prefix expression.

Method 2: Direct Dual-Stack Infix to Prefix Parsing

Using an **Operator Stack** and an **Operand Stack** simultaneously:

- When an operand is scanned, push it onto the Operand Stack.
- When an operator is popped along with two operands $\texttt{op}_1$ and $\texttt{op}_2$, concatenate operator $+ \texttt{op}_1 + \texttt{op}_2$ and push the concatenated prefix string back onto the Operand Stack.

9.2 Expression Trees & Structural Syntax Representation

An **Expression Tree** is a binary tree representation of an arithmetic formula:

- **Leaf Nodes:** Always correspond strictly to operands (variables or constants).
- **Internal Nodes:** Always correspond to operators (`+`, `-`, `*`, `/`, `**`).
- **Traversals and Notations Correspondence:**
  1. **In-Order Traversal (Left, Root, Right):** Produces the conventional Infix expression. Grouping parentheses must be added at each parent node to preserve evaluation precedence.
  2. **Pre-Order Traversal (Root, Left, Right):** Produces the Prefix (Polish) expression.
  3. **Post-Order Traversal (Left, Right, Root):** Produces the Postfix (Reverse Polish) expression.

Constructing an Expression Tree from Postfix

1. Initialize an empty stack of tree nodes.
2. Read the postfix expression token by token from left to right:
   - If token is an operand: Create a new single-node tree and push its pointer onto the stack.
   - If token is an operator: Create a new operator node. Pop the top node from stack and attach as its **Right Child**. Pop the next node and attach as its **Left Child**. Push the root operator node back onto the stack.
3. At termination, the remaining node on the stack is the root of the complete Expression Tree.

9.3 CPython Dynamic Array Growth & Amortized Complexity

A common conceptual question in advanced computer science papers is analyzing why Python `list.append()` achieves $O(1)$ amortized time despite the physical constraints of RAM.

1. The CPython Over-Allocation Equation

When a Python list exhausts its allocated buffer capacity, CPython does not allocate space for merely one additional element. Instead, it over-allocates contiguous memory using the formula:

```
new_allocated = (size_t)newsize + (newsize >> 3) + (newsize < 9 ? 3 : 6);
```

This expands the buffer by roughly 12.5% (a geometric expansion factor of approximately 1.125 $\times$).

2. Aggregate Amortized Proof

- Inserting N items requires occasional reallocations where all existing elements are copied to a new memory block.
- For N insertions, the total number of element copies across all resizing events forms a bounded geometric series:

$$\text{Total Copies} \leq \frac{N}{2} + \frac{N}{4} + \frac{N}{8} + \ldots \leq N$$

- The total execution time for N append operations is $O(N)$.
- Amortized cost per individual push:

$$\text{Amortized Cost} = \frac{O(N)}{N} = O(1)$$

WORKED EXAMPLE 6: MONOTONIC STACK FOR NEXT GREATER ELEMENT (NGE)

Given an integer array `nums = [4, 5, 2, 25]`, find the Next Greater Element for each item in strictly $O(N)$ linear time. The Next Greater Element for an element x is the first greater element to its right in the array. If no greater element exists, output `-1`.

Python Implementation:

```python
def next_greater_elements(nums):
    n = len(nums)
    result = [-1] * n
    stack = []  # Monotonic decreasing stack storing indices

    for i in range(n):
        while stack and nums[i] > nums[stack[-1]]:
            prev_idx = stack.pop()
            result[prev_idx] = nums[i]
        stack.append(i)

    return result

print(next_greater_elements([4, 5, 2, 25]))  # Output: [5, 25, 25, -1]
```

Complexity Analysis: Although a nested `while` loop is present, each index is pushed onto the stack exactly once and popped at most once across the entire traversal. Total runtime is strictly $O(N)$, outperforming the naive $O(N^2)$ brute-force approach.

WORKED EXAMPLE 7: DIRECT PREFIX EVALUATION TRACE

Evaluate the prefix expression `- + * 4 5 / 16 4 7` step-by-step using an operand stack with right-to-left scanning.

Trace:

1. Scan tokens from right to left: `7`, `4`, `16`, `/`, `5`, `4`, `*`, `+`, `-`
2. Push `7`, `4`, `16` → Stack: `[7, 4, 16]`
3. Operator `/`: Pop `op1 = 16`, `op2 = 4`. Compute $\frac{16}{4} = 4.0$. Push `4.0` → Stack: `[7, 4.0]`
4. Push `5`, `4` → Stack: `[7, 4.0, 5, 4]`
5. Operator `*`: Pop `op1 = 4`, `op2 = 5`. Compute $4 \times 5 = 20.0$. Push `20.0` → Stack: `[7, 4.0, 20.0]`
6. Operator `+`: Pop `op1 = 20.0`, `op2 = 4.0`. Compute $20.0 + 4.0 = 24.0$. Push `24.0` → Stack: `[7, 24.0]`
7. Operator `-`: Pop `op1 = 24.0`, `op2 = 7`. Compute $24.0 - 7 = 17.0$. Push `17.0` → Stack: `[17.0]`

Final Result: `17.0`

Master Mindmap: Stack Data Structure



Figure 3: Figure 3: Master Synthesis Mindmap: Structural framework of Unit 2 Stack Data Structure & Algorithms.

OurMocks

High-Yield Cheatsheet & Exam Traps

MASTER STACK ADT & EXPRESSIONS REVISION CHEATSHEET

- **LIFO Discipline:** Elements enter and depart strictly via a single logical endpoint (Top). Base elements remain inaccessible until all overlying elements are popped.
- **Primitive Operations:** `push(x)` ($O(1)$), `pop()` ($O(1)$), `peek()` ($O(1)$), `is_empty()` ($O(1)$), `size()` ($O(1)$).
- **Underflow vs Overflow:** Underflow occurs on `pop()`/`peek()` when Top $= -1$ (`len == 0`). Overflow occurs on bounded stacks when Top $=$ Max $- 1$.
- **List Implementation:** In standard Python, use `list.append(x)` to push and `list.pop()` to pop. Index `-1` is the Top of Stack ($O(1)$ amortized).
- **collections.deque:** Doubly-linked block buffer providing strictly guaranteed $O(1)$ push/pop with zero dynamic resizing spikes.
- **Infix, Prefix, Postfix:** Infix ($A + B$, human readable, needs parentheses); Prefix ($+AB$, Polish notation); Postfix ($AB+$, Reverse Polish notation, stack-friendly).
- **Shunting-Yard Invariants:** Operands go straight to output. Left parenthesis `(` pushed to stack. Right parenthesis `)` pops operators until matching `(` is discarded.
- **Operator Precedence:** Parentheses (Highest) > `**` (Level 3, Right-to-Left) > `* / // %` (Level 2, Left-to-Right) > `+ -` (Level 1, Left-to-Right).
- **Postfix Evaluation:** Operand stack; on operator, pop op_2 then op_1; compute $\text{op}_1 \;\; \text{op} \;\; \text{op}_2$; push result back to stack.
- **Call Stack & Recursion:** PVM activation records allocated on hardware call stack; default depth 1000 (`sys.getrecursionlimit()`), triggers `RecursionError`.

THE SEVEN FATAL CUET TRAPS IN STACK DATA STRUCTURES

1. **The Index 0 Stack Inversion Trap:** Implementing stack push via `insert(0, x)` and pop via `pop(0)` shifts all elements in memory, ballooning execution from $O(1)$ to catastrophic $O(N)$ linear time! Always use `append()` and `pop()`.
2. **The Postfix Operand Pop Inversion Trap:** In postfix evaluation, the first popped element is op_2 (Right Operand) and the second is op_1 (Left Operand). Calculating $\text{op}_2 - \text{op}_1$ or $\frac{\text{op}_2}{\text{op}_1}$ completely inverts signs and division quotients!
3. **The Right-to-Left Exponentiation Trap:** `2 ** 3 ** 2` evaluates as `2 ** (3 ** 2) = 2 ** 9 = 512`, NOT `(2 ** 3) ** 2 = 64`. In infix-to-postfix conversion, an incoming `**` does **not** pop an existing `**` from the operator stack because of right-associativity.
4. **The Unchecked Underflow Trap:** Calling `pop()` or accessing `stack[-1]` without first validating `if not stack:` or `len(stack) == 0` raises an unhandled `IndexError: pop from empty list`.
5. **Parentheses in Postfix Output Trap:** Grouping parentheses `(` and `)` are internal parsing tokens and **never** appear in the converted postfix or prefix string!
6. **The `is_empty` Falsy Pitfall:** An empty stack evaluates as `False` in Python conditional statements (`if not stack:`). Do not confuse an empty stack with a stack containing a single element with value `0` or `False` (which has `len(stack) == 1`).
7. **Recursion Depth Limit Trap:** Assuming recursive stack traversal can run infinitely. Unbounded recursion raises `RecursionError` at frame 1000 by default; production algorithms require iterative stack loops for large inputs.

CUET UG Exam-Style Practice Questions (15 MCQs)

Q1. Which of the following data structures operates strictly on the Last-In, First-Out (LIFO) access principle?

(A) Stack **(B)** Queue
(C) Binary Search Tree **(D)** Linear Array

Sol. Q1. A Stack is defined as a restricted linear data structure where elements are inserted and removed exclusively from one logical endpoint termed the Top, enforcing the Last-In, First-Out (LIFO) protocol.
Answer: (A)

Q2. What is the worst-case time complexity of pushing an element onto a stack implemented using Python's `collections.deque`?

(A) $O(N)$ **(B)** $O(1)$
(C) $O(\log N)$ **(D)** $O(N^2)$

Sol. Q2. `collections.deque` is implemented internally as a doubly-linked list of fixed-size blocks. Appending an element to either end requires neither array resizing nor memory copying, guaranteeing strictly $O(1)$ worst-case time.
Answer: (B)

Q3. Convert the infix expression `A + B * C - D / E` into its corresponding Postfix (Reverse Polish) notation:

(A) `A B + C * D E / -` **(B)** `A B C * + D - E /`
(C) `A B C * + D E / -` **(D)** `+ A * B C - / D E`

Sol. Q3. Step-by-step Shunting-Yard conversion:

1. `*` has highest precedence: `B * C` → `B C *`.
2. `+` precedes `-` due to left-associativity: `A + (B C *)` → `A B C * +`.
3. `/` evaluated next: `D / E` → `D E /`.
4. Subtraction evaluated: `(A B C * +) - (D E /)` → `A B C * + D E / -`.

Answer: (C)

Q4. What is the numerical result of evaluating the postfix expression `10 5 2 * + 8 4 / -` using a stack?

(A) 16 **(B)** 22
(C) 14 **(D)** 18

Sol. Q4. Tracing operand stack:

1. Push `10`, `5`, `2` → Stack: `[10, 5, 2]`.
2. Token `*`: pop `2`, pop `5` → $5 \times 2 = 10$. Push `10` → Stack: `[10, 10]`.
3. Token `+`: pop `10`, pop `10` → $10 + 10 = 20$. Push `20` → Stack: `[20]`.
4. Push `8`, `4` → Stack: `[20, 8, 4]`.
5. Token `/`: pop `4`, pop `8` → $\frac{8}{4} = 2$. Push `2` → Stack: `[20, 2]`.
6. Token `-`: pop `2`, pop `20` → $20 - 2 = 18$. Push `18`. Final Result: 18.

Answer: (D)

Q5. If an inexperienced programmer implements a stack using a Python list where `push(x)` is written as `s.insert(0, x)` and `pop()` as `s.pop(0)`, what is the time complexity of each push and pop operation?

(A) $O(N)$ linear time for both push and pop **(B)** $O(1)$ constant time for both push and pop
(C) $O(1)$ for push and $O(N)$ for pop **(D)** $O(N)$ for push and $O(1)$ for pop

Sol. Q5. Because Python lists are contiguous arrays, inserting or deleting at index 0 forces the runtime to shift all existing N elements by one position to the right (on insert) or left (on delete), degrading both operations to $O(N)$ linear time.
Answer: (A)

CUET UG Exam-Style Practice Questions (Part 2)

Q6. What is the result of evaluating the infix expression `2 ** 3 ** 2` and what is the associativity rule governing the exponentiation operator in Python?

(A) 64, Left-to-Right
(B) 512, Right-to-Left
(C) 64, Right-to-Left
(D) 512, Left-to-Right

Sol. Q6. In Python and mathematical convention, the exponentiation operator ** has **Right-to-Left** associativity. Therefore, `2 ** 3 ** 2` is grouped as `2 ** (3 ** 2) = 2 ** 9 = 512`.
Answer: (B)

Q7. During postfix evaluation using an operand stack, when a binary operator is popped, which of the following represents the correct evaluation formula?

(A) res = op2 op op1, where op2 was popped second
(B) res = op1 op op2, where op1 was popped first
(C) res = op1 op op2, where op2 was popped first and op1 second
(D) res = op2 op op1, where op1 was popped first and op2 second

Sol. Q7. In postfix evaluation, because of LIFO order, the first element popped from the stack is the right-hand operand (op2), and the second element popped is the left-hand operand (op1). The arithmetic calculation is strictly op1 op op2.
Answer: (C)

Q8. Which of the following built-in Python exceptions is raised when `pop()` is executed on an empty Python list representing a stack?

(A) `ValueError`
(B) `KeyError`
(C) `OverflowError`
(D) `IndexError`

Sol. Q8. Invoking `pop()` on an empty list `[]` raises `IndexError: pop from empty list`, indicating a stack underflow condition in Python list implementations.
Answer: (D)

Q9. Which of the following applications does NOT fundamentally rely on a Stack data structure?

(A) Breadth-First Search (BFS) graph shortest path traversal
(B) Depth-First Search (DFS) graph traversal
(C) Syntax parsing and balanced delimiter verification in compilers
(D) Undo and Redo operation buffers in word processors

Sol. Q9. Breadth-First Search (BFS) fundamentally relies on a **Queue** (FIFO) to explore nodes layer by layer. DFS, delimiter matching, and undo/redo buffers all fundamentally utilize Stacks (LIFO).
Answer: (A)

Q10. What is the maximum recursion depth permitted by default in the standard CPython runtime before a `RecursionError` is raised?

(A) 100
(B) 1000
(C) 10000
(D) Unlimited

Sol. Q10. The standard default recursion depth in CPython is 1000 stack frames. It can be queried via `sys.getrecursionlimit()` and modified using `sys.setrecursionlimit()`.
Answer: (B)

CUET UG Exam-Style Practice Questions (Part 3)

Q11. Consider a stack of maximum capacity 5 initially empty. The following operations are executed: `push(10)`, `push(20)`, `pop()`, `push(30)`, `push(40)`, `peek()`, `push(50)`. What is the current element at the top of the stack and what is the stack size?

(A) Top: 40, Size: 3
(B) Top: 50, Size: 5
(C) Top: 50, Size: 4
(D) Top: 30, Size: 4

Sol. Q11. Tracing state:

1. `push(10)` → `[10]`
2. `push(20)` → `[10, 20]`
3. `pop()` returns `20` → `[10]`
4. `push(30)` → `[10, 30]`
5. `push(40)` → `[10, 30, 40]`
6. `peek()` returns `40` (stack unchanged) → `[10, 30, 40]`
7. `push(50)` → `[10, 30, 40, 50]`.

Top element is `50`, and total size is `4`.

Answer: (C)

Q12. In the Shunting-Yard algorithm converting infix to postfix, what action is performed when a right parenthesis `)` is encountered in the token stream?

(A) Push `)` directly onto the operator stack
(B) Append `)` immediately to the postfix output expression
(C) Clear the entire operator stack and abort execution
(D) Pop operators from stack and append to output until matching `(` is popped and discarded

Sol. Q12. When a closing right parenthesis `)` is encountered, all operators on the stack are popped and appended to the postfix output until the matching left parenthesis `(` is reached. The `(` is popped from the stack and discarded; neither `(` nor `)` appears in the postfix string.

Answer: (D)

Q13. Given the prefix expression `+ * 2 3 - 8 4`, what is its numerical evaluated value?

(A) 10
(B) 14
(C) 8
(D) 12

Sol. Q13. Scanning prefix from right to left:

1. Operands `4`, `8` → Stack: `[4, 8]`
2. Operator `-`: pop `op1 = 8`, `op2 = 4` → $8 - 4 = 4$. Push `4` → Stack: `[4]`
3. Operands `3`, `2` → Stack: `[4, 3, 2]`
4. Operator `*`: pop `op1 = 2`, `op2 = 3` → $2 \times 3 = 6$. Push `6` → Stack: `[4, 6]`
5. Operator `+`: pop `op1 = 6`, `op2 = 4` → $6 + 4 = 10$. Push `10`. Final Result: 10.

Answer: (A)

Q14. Which module in Python standard library provides a thread-safe LIFO stack implementation intended for multi-threaded consumer-producer environments?

(A) `collections.deque`
(B) `queue.LifoQueue`
(C) `threading.Stack`
(D) `asyncio.LifoQueue`

Sol. Q14. `queue.LifoQueue` is the standard library class implementing a synchronized, thread-safe LIFO queue (stack) using locking primitives to prevent concurrent race conditions.

Answer: (B)

Q15. What is the auxiliary space complexity required to convert an infix expression containing N characters into postfix notation using a stack?

(A) $O(1)$ constant space
(B) $O(N^2)$ quadratic space
(C) $O(N)$ linear space
(D) $O(\log N)$ logarithmic space

Sol. Q15. In the worst case (e.g. an expression with many nested operators and parentheses `((((A+B)*C)-D)/E)`), the operator stack stores up to $O(N)$ tokens simultaneously. Hence, auxiliary space complexity is $O(N)$.

Answer: (C)